CS4745 Assignment 2

Antoine Giroux 3616164

2026/04/02

Design Summary

Adaptive Integration Method

For any interval [a,b] this algorithm first calculates the Simpson's estimate and compares it to the sum of the estimate of the two halves, if the difference exceeds the tolerance then it recursively subdivides the regions. This allows high precision in high-frequency regions while skipping unecessary calculations in low-frequency regions.

Task Granularity

The interval [0,1] is divided into K=64 initial base intervals, I chose 64 because it ensures that each process has multiple tasks to do even with 8 processes. 64 also felt right because if it was too small then a worker might finish all of its work before other workers are finished, but if it was too large then the master process would become the bottleneck due to the ammount of messages it would be receiving.

Master/Worker Communication Design

Master

The master maintains a queue of tasks, of size 64 which is enough for each worker to have work, it then listens for two types of messages from workers: - RESULT: The worker has finished and gives the Master a value to be added to the global sum - WORK: A worker has encountered a high error region and is adding more work to the queue.

Worker

The worker listens for tasks, performing one level of the adaptive split. If more recursion is needed it sends the right half back to the master as more work to be given to idle processes and processes the left half itself through recursion, aiming to minimize the ammount of time a processor is idle.

Termination Strategy

Termination is not really simpel as the Master doesnt know ahead of time how many tasks the workers might generate. To ensure that we only terminate at the correct time the master maintains a counter of tasks that have been sent out to the workers but not yet returned, Stops only when the queue is empty and the set of active processes is zero and to stop the while loop in the worker sends out a STOP message to each worker afterwards.

Result Table

Performance Results

Performance Results (Tolerance: 1e-6)

Function	Serial (M0)	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.000052	0.000117	0.000773	0.000140	0.000628	0.000130	0.000320
Func 1	0.000008	0.000116	0.000730	0.000072	0.000467	0.000162	0.000574
Func 2	0.000416	0.000194	0.002891	0.000265	0.002318	0.000109	0.003963

Performance Results (Tolerance: 1e-8)

Function	Serial (M0)	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.000021	0.000069	0.000940	0.000068	0.000660	0.000098	0.000762
Func 1	0.000011	0.000079	0.001046	0.000141	0.000631	0.000277	0.000431
Func 2	0.001747	0.000425	0.003814	0.000251	0.002282	0.000332	0.001854

Speedup Tables

Performance Results (Tolerance: 1e-6)

Function	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.44	0.07	0.37	0.08	0.40	0.16
Func 1	0.07	0.01	0.11	0.02	0.05	0.01
Func 2	2.14	0.14	1.57	0.18	3.82	0.10

Performance Results (Tolerance: 1e-8)

Function	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.30	0.02	0.31	0.03	0.21	0.03
Func 1	0.14	0.01	0.08	0.02	0.04	0.03
Func 2	4.11	0.46	6.96	0.77	5.26	0.94

Efficiency Tables

Performance Results (Tolerance: 1e-6)

Function	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.22	0.03	0.09	0.02	0.05	0.02
Func 1	0.03	0.01	0.03	0.01	0.01	0.00
Func 2	1.07	0.07	0.39	0.04	0.48	0.01

Performance Results (Tolerance: 1e-8)

Function	P=2 Static	P=2 Dynamic	P=4 Static	P=4 Dynamic	P=8 Static	P=8 Dynamic
Func 0	0.15	0.01	0.08	0.01	0.03	0.00
Func 1	0.07	0.01	0.02	0.00	0.00	0.00
Func 2	2.06	0.23	1.74	0.19	0.66	0.12

Plots

Discussion

Why does static decomposition perform poorly for adaptive workloads?

Static decomposition performs poorly as operates under the assumption that each region requires the same ammount of work, where some regions would require a lot of recursive calls while some would not, leading to poor CPU utilization as some processes would be doing most of the work.

How does dynamic task creation improve load balance?

Dynamic scheduling solves this by treating the integration intervals as a queue of work to be handed to processes instead of assigning each process a portion of

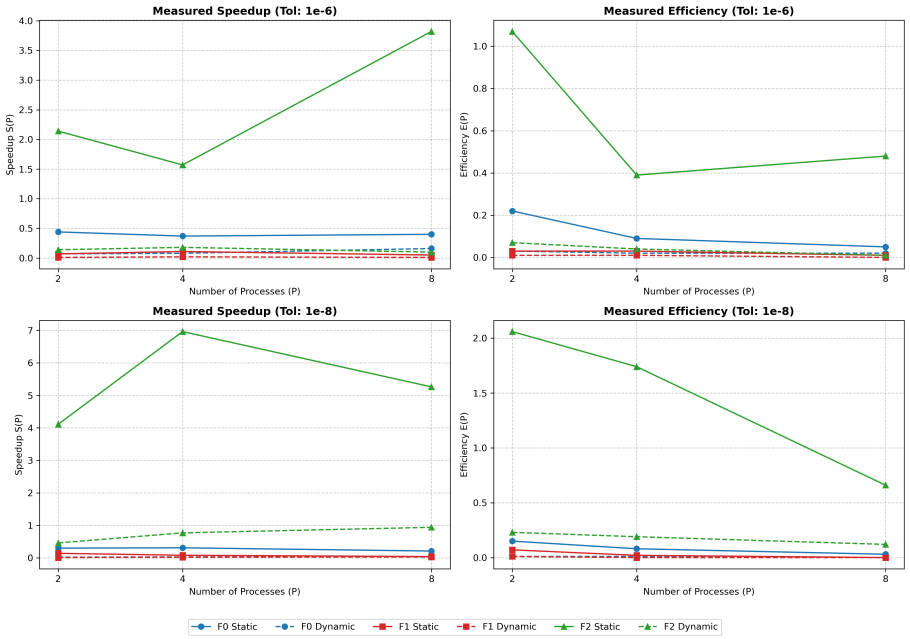


Figure 1: Performance Plots for Adaptive Integration

the interval. This allows workers to enqueue more work to be assigned to idle threads when under load, thus distributing the work among all available cores.

What is the cost of dynamic scheduling in MPI?

The greatest cost is Communication latency and the Overhead from the Master. Every time a worker sends out a result or receives a task, that data is sent over the network. If the tolerance is too high, then more time could be spent talking with the Master than is being spent on computing the result.

Which function benefited most from dynamic scheduling, and why?

Function 2 benefited the most from dynamic scheduling as they spend more time calculating then it does communicating, as opposed to function 0 and 1 who finish fast enough that the overhead from communicating is significant enough to slow down.